Protected Mode, Prefilters, and User Memory Bank

This note documents how the app implements Impinj **Protected Mode** (protect / unprotect), how **prefilters** are used to scope operations to specific tags, and how the **USER / RESERVED memory banks** are read and written. All code lives in:

- `app/src/main/java/com/example/newgen2xplay/ui/customImpinj/TagProtectFragment.java`
- `app/src/main/java/com/example/newgen2xplay/ui/Filters/PreFiltersFragment.java`
- `app/src/main/java/com/example/newgen2xplay/ui/Filters/PreFiltersViewModel.java`
- `app/src/main/java/com/example/newgen2xplay/ui/Inventory/InventoryViewModel.java`
- `app/src/main/java/com/example/newgen2xplay/RFIDHandler.java`

The Zebra RFID SDK entry points are exposed through two shared statics created once at connect time in RFIDHandler.java:

```
public static ImpinjExtensions impinjExtensions;      // Impinj Monza custom commands
// ...
impinjExtensions = new ImpinjExtensions(mConnectedRfidReader);  // RFIDHandler.java#L374
```

`mConnectedRfidReader.Actions.TagAccess` and `mConnectedRfidReader.Actions.PreFilters` are the standard Gen2 access/filter APIs.

---

1. Impinj Monza "Protected Mode" concept

Impinj Monza tags support a vendor feature called **Protected Mode**. When a tag is put into Protected Mode with a PIN/password:

- The tag becomes **invisible** to normal inventory (it does not respond to standard `Select`/`Query`).
- The reader can only see it again after sending a **short-range / visibility unlock** command with the correct password (`enableTagVisibility`).
- Protection can be permanently removed by **unprotecting** the tag with the password (`disableTagProtection`).

The UI exposes six operations via the spinner (`R.array.Protect_op_type_array` in `strings.xml`):

Spinner item	Meaning	Required inputs
`Protect`	Put tag into Protected Mode	TagID + Password
`Unprotect`	Remove Protected Mode from tag	TagID + Password
`GetPassword`	Read the access password (RESERVED bank)	TagID
`SetPassword`	Write the access password (RESERVED bank)	TagID + Password
`Enable Inventory Of Protected Tags`	Temporarily make protected tags visible	Password
`Clear Protected Mode Configuration`	Stop reading protected tags again	Password

All RFID work runs on a single-thread `ExecutorService` (never the UI thread); results are marshalled back with a main-looper `Handler`. See TagProtectFragment.java.

---

2. Protected mode — `Protect`

Protecting a tag is a single Impinj call. The password is an 8-char / 32-bit hex PIN (see `password_hint` in strings.xml).

```
// TagProtectFragment.handlePerformOp(...) - "Protect" branch
if (password.isEmpty() || tagId.isEmpty()) {
    mainHandler.post(() -> Toast.makeText(requireContext(),
        "TagID and Password cannot be empty", Toast.LENGTH_SHORT).show());
    return;
}
try {
    impinjExtensions.enableTagProtection(tagId, password, null);
    mainHandler.post(() -> Toast.makeText(requireContext(),
        "Protect Success", Toast.LENGTH_SHORT).show());
} catch (OperationFailureException e) {
    // e.getVendorMessage() carries the tag-level error (wrong PIN, no tag, etc.)
    mainHandler.post(() -> Toast.makeText(requireContext(),
        "Protect Failed " + e.getVendorMessage(), Toast.LENGTH_SHORT).show());
    return;
} catch (InvalidUsageException e) {
    mainHandler.post(() -> Toast.makeText(requireContext(),
        "Protect Failed " + e.getMessage(), Toast.LENGTH_SHORT).show());
    return;
}
```

Notes: - `enableTagProtection(tagId, password, antennaInfo)` — the first argument is the EPC used as the singulation match, so only that tag is affected. - `OperationFailureException.getVendorMessage()` is the meaningful message for tag-side failures; `InvalidUsageException.getMessage()` is for bad arguments/SDK misuse. The code deliberately separates the two.

After this call the tag will no longer appear in a normal inventory.

---

3. Unprotected mode — `Unprotect`

Unprotecting reverses Protected Mode. Because a protected tag may still have a prefilter targeting it, the code first clears prefilters, then unprotects.

```
// TagProtectFragment.handlePerformOp(...) - "Unprotect" branch
try {
    mConnectedRfidReader.Actions.PreFilters.deleteAll();          // clear any state-aware filters
    impinjExtensions.disableTagProtection(tagId, password, null, (short) 1);
    mainHandler.post(() -> Toast.makeText(requireContext(),
        "Unprotect Success", Toast.LENGTH_SHORT).show());
} catch (OperationFailureException e) {
    mainHandler.post(() -> Toast.makeText(requireContext(),
        "Unprotect Failed " + e.getVendorMessage(), Toast.LENGTH_SHORT).show());
    e.printStackTrace();
    return;
} catch (InvalidUsageException | IllegalStateException | NumberFormatException e) {
    mainHandler.post(() -> Toast.makeText(requireContext(),
        "Unprotect Failed " + e.getMessage(), Toast.LENGTH_SHORT).show());
    e.printStackTrace();
    return;
}
```

Notes: - `disableTagProtection(tagId, password, antennaInfo, antennaID)` needs the same password that was used to protect the tag. - The extra `(short) 1` is the antenna ID. - `deleteAll()` first avoids a stale prefilter accidentally hiding the tag during the unprotect singulation.

4. Temporarily seeing protected tags — visibility toggle

Two operations change **reader-side visibility** without permanently changing the tag's protection state:

- `Enable Inventory Of Protected Tags` → `enableTagVisibility(password, antenna)`
- `Clear Protected Mode Configuration` → `disableTagVisibility(password, antenna)`

```
// TagProtectFragment.handleEnableVisibilityChecked()
executor.execute(() -> {
    try {
        mConnectedRfidReader.Actions.PreFilters.deleteAll();
        impinjExtensions.enableTagVisibility(password, (short) 1);
        mainHandler.post(() -> Toast.makeText(requireContext(),
            "Enable Visibility Success", Toast.LENGTH_SHORT).show());
    } catch (OperationFailureException | InvalidUsageException e) {
        mainHandler.post(() -> Toast.makeText(requireContext(),
            "Enable Visibility Failed: " + e.getMessage(), Toast.LENGTH_SHORT).show());
        e.printStackTrace();
    }
});
```

```
// TagProtectFragment.handleDisableVisibilityChecked()
executor.execute(() -> {
    try {
        mConnectedRfidReader.Actions.PreFilters.deleteAll();
        impinjExtensions.disableTagVisibility(password, (short) 1);
        mainHandler.post(() -> Toast.makeText(requireContext(),
            "Disable Visibility Success", Toast.LENGTH_SHORT).show());
    } catch (OperationFailureException | InvalidUsageException | IllegalStateException e) {
        mainHandler.post(() -> Toast.makeText(requireContext(),
            "Disable Visibility Failed: " + e.getMessage(), Toast.LENGTH_SHORT).show());
        e.printStackTrace();
    }
});
```

Difference vs. Protect/Unprotect:

- `enableTagVisibility` / `disableTagVisibility` change whether the **reader** will surface already-protected tags in inventory. `disableTagVisibility` ("Clear Protected Mode Configuration") means the reader stops reading protected tags — the tag itself remains protected. This is stated in the in-app help dialog (`showRequiredValuesDialog()`).
- `enableTagProtection` / `disableTagProtection` change the **tag's** stored protection state.

---

5. Access password in the RESERVED bank — `GetPassword` / `SetPassword`

The Monza access PIN lives in the **RESERVED** memory bank (bank 0). The access password occupies words 2–3 (offset 2, count 2 words = 32 bits). These two operations use the standard `TagAccess` read/write API, not the Impinj extension.

GetPassword (read RESERVED bank)

```
// TagProtectFragment.handlePerformOp(...) - "GetPassword" branch
TagAccess tagAccess = new TagAccess();
TagAccess.ReadAccessParams readAccessParams = tagAccess.new ReadAccessParams();
```

```
readAccessParams.setAccessPassword(DEFAULT_ACCESS_PASSWORD);       // 0 = open access (unlocked tag)
readAccessParams.setCount(2);                                      // 2 words = 32-bit password
readAccessParams.setMemoryBank(MEMORY_BANK.MEMORY_BANK_RESERVED); // bank 0
readAccessParams.setOffset(2);                                     // word offset 2 (access pwd)

// If the EPC pattern is short enough to fit a prefilter (<= 24 hex chars / 96 bits),
// let the SDK build a prefilter automatically to single out this tag.
if (tagId.length() <= 24) {
    isPrefilterRead.set(true);
}

tagData.set(mConnectedRfidReader.Actions.TagAccess.readWait(
        tagId, readAccessParams, null, isPrefilterRead.get()));

// On the UI thread:
if (tagData.get().getOpStatus() == ACCESS_OPERATION_STATUS.ACCESS_SUCCESS) {
    binding.etPassword.setText(tagData.get().getMemoryBankData());   // hex string
}
```

SetPassword (write RESERVED bank)

```
// TagProtectFragment.handlePerformOp(...) - "SetPassword" branch
TagAccess tagAccess = new TagAccess();
TagAccess.WriteAccessParams writeAccessParams = tagAccess.new WriteAccessParams();
String writeData = password;                                       // 8 hex chars = 32 bits
writeAccessParams.setAccessPassword(DEFAULT_ACCESS_PASSWORD);      // 0 = open access (unlocked tag)
writeAccessParams.setMemoryBank(MEMORY_BANK.MEMORY_BANK_RESERVED); // bank 0
writeAccessParams.setOffset(2);                                    // access-password words
writeAccessParams.setWriteData(writeData);
writeAccessParams.setWriteDataLength(2);                           // 2 words

if (tagId.length() <= 24) {
    isPrefilterWrite.set(true);
}

mConnectedRfidReader.Actions.TagAccess.writeWait(
        tagId, writeAccessParams, null, writeTagData,
        isPrefilterWrite.get(), false);

if (writeTagData.getOpStatus() == ACCESS_OPERATION_STATUS.ACCESS_SUCCESS) {
    // password written
}
```

Key params: - `MEMORY_BANK.MEMORY_BANK_RESERVED` (bank 0) holds **kill password** (words 0–1) and **access password** (words 2–3). Offset 2 targets the access password. - `readWait(epc, params, antennaInfo, useAutoPrefilter)` — the last boolean tells the SDK to build a prefilter from the EPC automatically so the access hits only the intended tag. - `writeWait(epc, params, antennaInfo, outTagData, useAutoPrefilter, doVerify)` — final boolean is a verify-after-write flag (here `false`).

The `tagId.length() <= 24` check is important: an EPC prefilter pattern must fit the 96-bit EPC field (24 hex chars). Longer identifiers cannot be used as an auto-prefilter, so `useAutoPrefilter` stays `false`.

---

6. Prefilters — scoping operations to selected tags

A **prefilter** (Gen2 `Select`) narrows which tags participate in the next inventory or access. The app uses them in two ways: a manual editor (`PreFiltersFragment`) and programmatic filters for quiet/access operations.

6.1 Memory-bank options in the UI

The prefilter editor lets the user choose the bank to match against (`R.array.pre_filter_memory_bank_array`):

```
<string-array name="pre_filter_memory_bank_array">
    <item>EPC</item>
    <item>TID</item>
    <item>USER</item>
</string-array>
```

The selected string is mapped to the SDK enum via `MEMORY_BANK.GetMemoryBankValue(...)`.

6.2 Building and saving a prefilter

```
// PreFiltersFragment - buttonSave click handler
PreFilters filters = new PreFilters();
PreFilters.PreFilter filter = filters.new PreFilter();
filter.setAntennaID((short) 1);
filter.setTagPattern(mask);                                    // hex mask to match
filter.setTagPatternBitCount(Integer.parseInt(length));        // how many bits of the mask matter
filter.setBitOffset(Integer.parseInt(pointer));                // where in the bank to start matching
filter.setMemoryBank(MEMORY_BANK.GetMemoryBankValue(
        binding.prefilterMembank.getSelectedItem().toString())); // EPC / TID / USER
filter.setFilterAction(FILTER_ACTION.FILTER_ACTION_STATE_AWARE);
filter.StateAwareAction.setTarget(
        TARGET.getTarget(binding.prefilterTarget.getSelectedItemPosition())); // SL / S0..S3
filter.StateAwareAction.setStateAwareAction(
        getStateAwareActionFromString(binding.prefilterAction.getSelectedItem().toString()));

viewModel.savePreFilter(filter);
```

The three matching parameters are the standard Gen2 `Select` fields: - **MemoryBank** — which bank the mask compares against (EPC / TID / USER). - **BitOffset (pointer)** — bit position in that bank where matching begins. For the EPC bank the first 32 bits are CRC+PC, so EPC data starts at offset 32. - **TagPatternBitCount (length)** — number of mask bits that must match.

6.3 Persisting via the ViewModel (off the UI thread)

```
// PreFiltersViewModel
public void savePreFilter(PreFilters.PreFilter filter) {
    new Thread(() -> {
        boolean isSuccess = false;
        try {
            mConnectedRfidReader.Actions.PreFilters.add(filter);
            isSuccess = true;
        } catch (Exception e) {
            isSuccess = false;
        }
        preFilterResult.postValue(isSuccess);
    }).start();
}
```

```
public void deletePreFilter(int index) { /* getPreFilter(index) then .delete(filter) */ }
public void deleteAllPreFilter()       { mConnectedRfidReader.Actions.PreFilters.deleteAll(); }
```

The SDK supports up to 4 prefilters (`R.array.pre_filter_index` = 1..4). Reading back an existing filter renders its raw pattern bytes as hex in `getPreFilter(index)`:

```
StringBuilder tagPattern = new StringBuilder();
for (byte b : preFilter.get().getTagPattern()) {
    tagPattern.append(String.format("%02X", b));
}
binding.etMask.setText(tagPattern);
```

6.4 Programmatic prefilters for quieting selected tags

For the "quiet selected tags" flow the app builds one `PreFilter` per selected EPC and adds them as a batch. This matches on the **EPC bank at offset 32**:

```
// InventoryViewModel.setPrefilterForQuietingTags(...)
PreFilters.PreFilter[] preFiltersArray = new PreFilters.PreFilter[length];
for (int i = 0; i < length; i++) {
    PreFilters preFilters = new PreFilters();
    PreFilters.PreFilter preFilter = preFilters.new PreFilter();
    preFilter.setAntennaID((short) 1);
    preFilter.setBitOffset(32);                    // skip CRC(16) + PC(16), start at EPC
    preFilter.setTagPatternBitCount(96);           // full 96-bit EPC
    preFilter.setTagPattern(arrayTags[i].getEPC());
    preFilter.setMemoryBank(MEMORY_BANK.MEMORY_BANK_EPC);
    preFilter.setFilterAction(FILTER_ACTION.FILTER_ACTION_STATE_AWARE);
    preFilter.StateAwareAction.setTarget(TARGET.TARGET_INVENTORIED_STATE_S3);
    preFilter.StateAwareAction.setStateAwareAction(STATE_AWARE_ACTION.STATE_AWARE_ACTION_INV_B);
    preFiltersArray[i] = preFilter;
}
mConnectedRfidReader.Actions.PreFilters.add(preFiltersArray, null);
```

Prefilters are always cleared with `PreFilters.deleteAll()` before starting a new operation (and once at connect time in RFIDHandler.java) so that a stale `Select` never leaks into the next inventory.

---

7. USER memory bank

The **USER** bank (bank 3) is user-defined tag storage. In this app the USER bank appears as a **prefilter target** — you can single out tags whose USER-bank contents match a mask:

```
// In the prefilter editor, choosing "USER" in the membank spinner resolves to
// MEMORY_BANK.MEMORY_BANK_USER via:
filter.setMemoryBank(MEMORY_BANK.GetMemoryBankValue("USER"));
```

Example: match all tags whose first USER word is `0x1234`:

Field	Value	Meaning
MemoryBank	`USER`	bank 3
BitOffset (pointer)	`0`	start of USER bank
TagPatternBitCount (length)	`16`	first word
TagPattern (mask)	`1234`	required contents

The four Gen2 memory banks and how the app addresses them:

Bank	Enum	Used for	Offsets seen in code
RESERVED (0)	`MEMORY_BANK_RESERVED`	kill / access passwords	offset 2, count 2 (access pwd)
EPC (1)	`MEMORY_BANK_EPC`	EPC identity	offset 32, 96-bit prefilter
TID (2)	`MEMORY_BANK_TID`	manufacturer ID	prefilter target only
USER (3)	`MEMORY_BANK_USER`	user data	prefilter target only

To add read/write of USER-bank *data* (not just filtering), reuse the `GetPassword` / `SetPassword` pattern from section 5 but set `params.setMemoryBank(MEMORY_BANK.MEMORY_BANK_USER)` and choose an `offset` / `count` (in 16-bit words) that fits the tag's USER capacity.

---

8. Cross-cutting conventions

- **Threading:** every reader call runs on an executor/worker thread; UI updates are posted back through `mainHandler` / `postValue`. Never call the SDK on the main thread.
- **Exception handling:** `OperationFailureException` → use `getVendorMessage()` (tag-side reason). `InvalidUsageException` → use `getMessage()` (argument/SDK misuse). The two are caught separately throughout `TagProtectFragment`.
- **Prefilter hygiene:** call `PreFilters.deleteAll()` before protect/unprotect and visibility operations so no stale `Select` interferes.
- **EPC-length guard:** auto-prefilter (`useAutoPrefilter = true`) is only enabled when `tagId.length() <= 24` (96-bit EPC limit).
- **Password format:** 8 hex chars = 32-bit access PIN (`R.string.password_hint`).